Here's a **cleanly structured and categorized** version of your questions, divided into **Programming Questions** and **Theoretical Questions**, with **reframed wording**, and **Java code provided** where applicable.

Programming Questions (Reframed + With Code)

1. Filter non-null values from a list

```
import java.util.*;
import java.util.stream.*;

public class FilterNonNull {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("A", null, "B", null, "C");
        List<String> filtered = list.stream()
                                    .filter(Objects::nonNull)
                                    .collect(Collectors.toList());
        System.out.println(filtered);
    }
}
```

---

2. Multithreaded program to avoid deadlock

```
public class DeadlockAvoidance {
    public static void main(String[] args) {
        Object lock1 = new Object();
        Object lock2 = new Object();

        Runnable task1 = () -> {
            synchronized (lock1) {
                System.out.println("Task1 acquired lock1");
                synchronized (lock2) {
                    System.out.println("Task1 acquired lock2");
                }
            }
        };

        Runnable task2 = () -> {
            synchronized (lock1) { // lock1 first, same order
                System.out.println("Task2 acquired lock1");
                synchronized (lock2) {
                    System.out.println("Task2 acquired lock2");
                }
            }
        };

        new Thread(task1).start();
```

```
        new Thread(task2).start();
    }
}
```

---

3. Return the customer name with the 2nd highest salary

```
import java.util.*;
import java.util.stream.*;

class Customer {
    String name;
    double salary;

    Customer(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}

public class SecondHighest {
    public static void main(String[] args) {
        List<Customer> customers = Arrays.asList(
            new Customer("A", 2000),
            new Customer("B", 3000),
            new Customer("C", 4000),
            new Customer("D", 1000)
        );

        String secondHighest = customers.stream()
            .sorted((c1, c2) -> Double.compare(c2.salary, c1.salary))
            .skip(1)
            .findFirst()
            .map(c -> c.name)
            .orElse("Not found");

        System.out.println("2nd Highest Salary: " + secondHighest);
    }
}
```

---

4. Calculate factorial of a number

```
public class Factorial {
    public static long factorial(int n) {
        if (n == 0) return 1;
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
```

```
        System.out.println(factorial(5));
    }
}
```

---

5. Compare two HashMaps (keys and objects), find common and different entries

(You mentioned HashMap with objects and FlatMap logic; assuming comparing maps with custom objects)

```
import java.util.*;

class Employee {
    int id;
    String name;

    // equals and hashCode overridden
}

public class MapCompare {
    public static void main(String[] args) {
        Map<Integer, String> map1 = Map.of(1, "A", 2, "B", 3, "C");
        Map<Integer, String> map2 = Map.of(2, "B", 3, "X", 4, "D");

        System.out.println("Common Keys: " + new
HashSet<>(map1.keySet()).retainAll(map2.keySet()));
        System.out.println("Different Keys: " + new
HashSet<>(map1.keySet()).removeAll(map2.keySet()));
    }
}
```

---

6. Remove duplicates from an array without using built-in methods

```
public class RemoveDuplicates {
    public static void main(String[] args) {
        int[] a = {1,1,2,2,2,3,4,4,5,5};
        int[] temp = new int[a.length];
        int size = 0;

        for (int i = 0; i < a.length; i++) {
            boolean found = false;
            for (int j = 0; j < size; j++) {
                if (a[i] == temp[j]) {
                    found = true;
                    break;
                }
            }
            if (!found) {
                temp[size++] = a[i];
```

```
            }
        }

        for (int i = 0; i < size; i++) {
            System.out.print(temp[i] + " ");
        }
    }
}
```

---

7. Find the frequency of a character in a string

```
public class CharFrequency {
    public static void main(String[] args) {
        String str = "programming";
        char target = 'g';
        int count = 0;

        for (char c : str.toCharArray()) {
            if (c == target) count++;
        }

        System.out.println("Frequency of '" + target + "': " + count);
    }
}
```

---

8. Create a copy of employee list with salary increment without modifying original

```
import java.util.*;
import java.util.stream.*;

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
}

public class CopyWithIncrement {
    public static void main(String[] args) {
        List<Employee> original = Arrays.asList(
            new Employee("John", 1000),
            new Employee("Jane", 2000)
        );

        List<Employee> updated = original.stream()
```

```
            .map(e -> new Employee(e.name, e.salary + 500))
            .collect(Collectors.toList());

        updated.forEach(e -> System.out.println(e.name + ": " +
e.salary));
    }
}
```

---

9. Generate all combinations of ["a", "b", "c"]

```
public class Combinations {
    public static void combine(String prefix, String[] arr, int start)
{
        for (int i = start; i < arr.length; i++) {
            String newPrefix = prefix + arr[i];
            System.out.println(newPrefix);
            combine(newPrefix, arr, i + 1);
        }
    }

    public static void main(String[] args) {
        String[] input = {"a", "b", "c"};
        combine("", input, 0);
    }
}
```

---

10. Sort employees by rating and name using Java 8

```
import java.util.*;

class Emp {
    String name;
    int rating;

    Emp(String name, int rating) {
        this.name = name;
        this.rating = rating;
    }
}

public class SortEmp {
    public static void main(String[] args) {
        List<Emp> list = Arrays.asList(
            new Emp("Alice", 4),
            new Emp("Bob", 4),
            new Emp("Charlie", 5)
        );
```

```
list.sort(Comparator.comparingInt(Emp::rating).thenComparing(Emp::name
));
        list.forEach(e -> System.out.println(e.name + " - " +
e.rating));
    }
}
```

---

11. Group employees by rating

```
import java.util.*;
import java.util.stream.*;

public class GroupEmp {
    public static void main(String[] args) {
        List<Emp> employees = Arrays.asList(
            new Emp("Alice", 4),
            new Emp("Bob", 5),
            new Emp("Charlie", 4)
        );

        Map<Integer, List<Emp>> grouped = employees.stream()
            .collect(Collectors.groupingBy(e -> e.rating));

        grouped.forEach((rating, empList) -> {
            System.out.println("Rating " + rating + ": " +
empList.size() + " employees");
        });
    }
}
```

Would you like a **separate PDF checklist** that includes:

- Programming questions + solutions
- Theoretical questions grouped by topic (Core Java, Collections, Spring Boot, Microservices, etc.)
- Real-time scenarios

Java Interview Preparation: Theoretical Questions & Real-Time Scenarios

---

Core Java

OOP Concepts:

- What is inheritance? Provide an example.
- What is polymorphism? How do abstract classes achieve it?
- What is encapsulation and data abstraction?
- Difference between abstract class and interface. Which one to use and when?

- Why is String immutable in Java?
- What is the String pool and how is memory managed for strings?
- What is the purpose of 'this' and 'super' keywords?

Exception Handling:

- What is the difference between checked and unchecked exceptions?
- Why should checked exceptions be handled explicitly?
- Explain the use of throw and throws.
- What is ConcurrentModificationException and how to avoid it?
- Is it possible to throw a runtime exception manually?

Multithreading & Concurrency:

- What is a thread and its lifecycle?
- What is the difference between sleep() and wait()?
- How does notify() and notifyAll() work?
- How can you make a class thread-safe?
- Difference between synchronizedMap() and ConcurrentHashMap?
- What is ExecutorService and how does it work?
- What is CompletableFuture?

Memory Management:

- Stack vs Heap memory
- When are static and instance variables initialized?
- What is Garbage Collection (GC)?
- How does JVM handle memory?
- Explain the types of ClassLoaders.
- What are the causes of OutOfMemoryError and how to handle it?

Java Basics:

- What is the purpose of final keyword?
- Why is Java considered platform-independent?
- What are functional interfaces and lambdas in Java 8?
- What is the use of Optional class in Java 8?

Collections Framework

Basics:

- Differences between ArrayList, LinkedList, HashSet, and TreeSet
- How does HashMap work internally?
- Can you store null keys and values in HashMap?
- Difference between HashMap and Hashtable

Advanced Concepts:

- When to use Map<Object, Object>?
- How do equals() and hashCode() impact collection behavior?
- What collection would you use to avoid duplicates in a custom class?
- Can you add set.add(1, 3)?
- When do you prefer ArrayList over LinkedList and vice versa?
- Explain the hierarchy of the Collection framework

Streams & Java 8:

- What is the purpose of the Stream API?
- How do map(), flatMap(), and filter() work?
- What is the use of the Collectors class?
- Grouping and sorting using streams

Spring Boot & Microservices

Spring Core:

- What is Inversion of Control (IoC)?
- Spring Bean lifecycle
- Difference between @Component, @Configuration, and @Bean
- How is data serialized between services?

Spring Boot:

- How is deployment done using Jenkins?
- What is the role of a controller?
- Explain how to configure basic controller-service-repo flow

Microservices:

- What is the microservices architecture?
- Difference between monolith and microservices
- Explain circuit breaker pattern
- How does API Gateway work?
- What is request routing?
- What is rate limiting and how is it different from load balancing?
- What are your roles and responsibilities in a microservices project?
- How are transactions managed across microservices (Saga, async/sync)?

Security:

- What is authentication vs authorization?
- How is security implemented in your application?
- How does LDAP work?

- What is multi-factor authentication?
- How to serialize and deserialize a bearer token?

System Design & Architecture

- Design an abstract factory pattern
- Design an immutable Employee class
- Design an ATM Machine (balance, withdraw, deposit)
- How does deployment work in Spring Boot + Jenkins?
- How do distributed systems communicate in your project?
- What is the role of the API Gateway?

Database & SQL

- Types of SQL Joins
- How do you design table schemas for customer-account relationship?
- Difference between NoSQL and relational DB
- How are DB connections handled in your project?
- Explain the flow of PATCH and PUT

Real-Time Scenarios & Debugging

- How do you handle OutOfMemoryError in production?
    - Temporary vs Permanent fixes
- How to process 1 million records efficiently from two feeds (stockId, price, exchange)?
    - What data structures would you use?
- How do you resolve dependency conflicts?
- Have you handled production issues? How?
- What dev tools are you using?
- Logging strategy used in the project?

Miscellaneous / Emerging Topics

- What is prompt engineering?
- How to give effective instructions to AI tools?
- What is a dark web?
- What is Gemini, ChatGPT, and how are they different?
- What is the role of Stack Overflow and GitHub in development?
- What is functional programming?
- What is the importance of big-O notation?